

DYNAMO: Dynamic Skill-Tool Evolution for Vision-Language Agents

Anonymous ACL submission

Abstract

Vision-language models (VLMs) remain brittle on visual reasoning tasks, and improving them typically requires either retraining or manual prompt/tool design. We ask a different question: can a frozen VLM agent use its own failed attempts as supervision for acquiring reusable capabilities? We present EVOAGENT, a training-free framework that turns diagnosable failures into two capability types: *skills*, structured reasoning procedures for task families where the visual evidence is available but the reasoning procedure is weak, and *tools*, executable Python programs for cases where the agent needs to crop, zoom, enhance, or otherwise transform the visual input. On each failure, EVOAGENT inspects the original image, intermediate tool outputs, and reasoning trace; decides whether the missing capability is cognitive, perceptual, or both; generates the appropriate capability; validates it; and accumulates it for future cases without updating model weights. Across completed from-scratch evaluations on ChartQA, MathVista, HRBench, and V*, skill-tool co-evolution improves direct inference in 21 of 23 model-benchmark cells, with an average gain of 4.8 accuracy points. In a provided-tool setting on GTA, mastery-guided capability use improves stronger backbones while showing that over-prescriptive tool-use policies can hurt less compatible models. Against RL-trained VTool-R1, skill evolution alone recovers 60.0–94.1% of the RL performance gap on 3B/7B chart and table reasoning splits. These results suggest that a substantial part of task-specific VLM adaptation can be recovered by failure-driven, in-context capability evolution.

1 Introduction

Large vision-language models (VLMs) have improved rapidly, but adapting them to a new visual reasoning distribution still usually requires either retraining or manual engineering. This is costly

in settings where model weights are unavailable, training data is small, or the failure modes are discovered only after deployment. A more practical alternative is to ask whether a frozen VLM agent can use its own failed attempts as supervision: if a case is answered incorrectly, can the agent inspect what went wrong, create a reusable capability, and apply that capability to future cases?

This question is motivated by a simple design distinction. Some failed attempts appear to be *cognitive*: the image contains the relevant evidence, but the agent needs a better procedure for reading, decomposing, or computing from it. Other failures appear to be *perceptual*: the agent needs a transformed view of the input before the relevant evidence is accessible at all, such as a crop, zoom, contrast enhancement, or chart rerendering. We do not assume that every benchmark failure belongs cleanly to one of these categories. Instead, we use the distinction as an operational decision for self-improvement: should a failure produce a reasoning skill, a visual tool, or both?

This distinction matters because the appropriate remedy depends on the bottleneck. Training-based methods can learn tool-use or reasoning behavior through supervised fine-tuning or reinforcement learning, but they require additional optimization and substantial compute. Manual prompt engineering can encode useful heuristics, but it is brittle and does not create executable visual operations. Text-only self-reflection can describe a mistake, but it cannot inspect image artifacts, decide whether the failure came from perception or reasoning, or generate image-processing code that changes what the model can see.

We therefore ask whether a frozen VLM agent can improve itself by turning failed attempts into reusable capabilities. Given a failed case, the agent should inspect the image and any intermediate visual artifacts, decide whether the missing capability is cognitive, perceptual, or both, generate the cor-

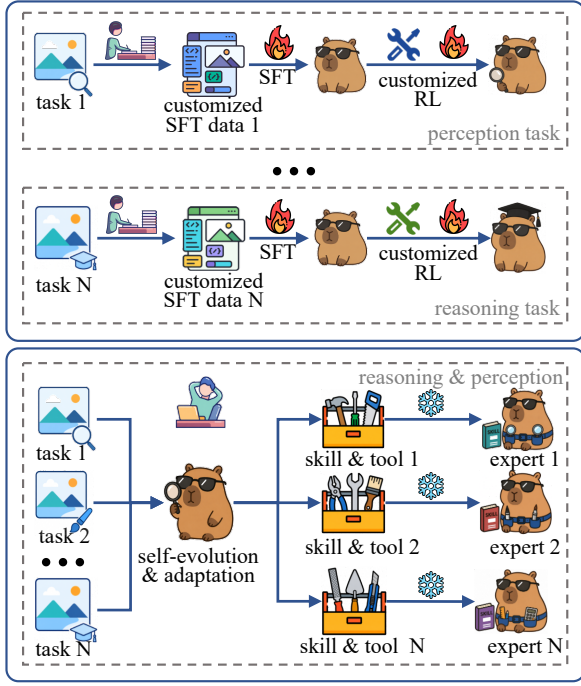


Figure 1: **Motivation for skill-tool evolution.** Failed VLM attempts can provide supervision for training-free adaptation. Some failures are better addressed by reusable reasoning skills, while others require executable visual tools that change what the model can inspect. Existing adaptation methods either rely on retraining, manual prompting, or text-only reflection. EVOAGENT instead diagnoses failures visually and evolves the capability type that matches the bottleneck.

responding capability, validate it, and reuse it on future cases. This leads to EVOAGENT, a training-free framework for dynamic skill-tool evolution. **Skills** are structured reasoning procedures, written as standard operating procedures for recurring problem classes. **Tools** are executable Python programs that manipulate visual inputs through operations such as cropping, zooming, contrast enhancement, or chart rerendering. The capability library grows only through validated failures; model weights remain frozen throughout.

We evaluate three questions. **(I) Can an agent evolve useful capabilities from scratch?** Starting with an empty library, EVOAGENT improves over direct inference in 21 of 23 completed model-benchmark cells across ChartQA, MathVista, HRBench, and V*, with gains varying by benchmark and capability type. **(II) Can an agent learn when to use provided tools?** On GTA, mastery-guided capability use improves GPT-5.4 and Gemini3Flash across step-level and answer metrics, while a regression on GPT-4o reveals

that tool-use policies must match the backbone’s instruction-following capacity. **(III) How much of RL-trained tool reasoning can be recovered without retraining?** On VTool-R1-aligned chart and table splits, skill evolution alone recovers a large fraction of the RL gap for Qwen2.5-VL backbones, especially at 3B and 7B scale.

Contributions.

- (1) We formulate VLM self-improvement as **failure-driven capability evolution**: a frozen agent diagnoses each failure and accumulates reusable skills and tools without gradient updates.
- (2) We introduce an operational **cognitive-perceptual decision rule** for visual failure handling, connecting reasoning skills to strategy-oriented fixes and executable visual tools to perception-oriented fixes.
- (3) We present a validation and mastery mechanism that prevents generated capabilities from being promoted solely because they solve the triggering example; tools must also pass regression checks and acquire applicability SOPs.
- (4) We provide empirical evidence across from-scratch evolution, provided-tool mastery, and comparison to RL-trained VTool-R1, showing both the promise and the limits of training-free capability evolution.

Paper organisation. Section 2 positions EVOAGENT against self-improving agents and tool-augmented VLMs. Section 3 describes the evolution framework. Section 4 presents the experiments, and Sections 5–6 discuss limitations and conclude.

2 Related Work

Self-improving agents. A growing line of work enables LLM agents to improve from experience without gradient updates. Reflexion (Shinn et al., 2023) maintains a verbal reflection buffer that the agent can consult on future attempts, but reflections are case-specific and non-persistent: once the episode ends, the buffer resets, so no reusable capability accumulates. Voyager (Wang et al., 2024) and ExpeL (Zhao et al., 2024) build libraries of executable skills from experience in game and tool-use domains, demonstrating that cross-case

skill transfer is feasible and valuable. AutoManual (Chen et al., 2024) automatically constructs structured instruction manuals for agent behaviour, and EvolveR (Anonymous, 2025a) iteratively refines reasoning strategies via self-play. These methods operate exclusively in text, code-execution, or game domains; none generates executable *visual processing* tools, and none confronts the dual-modality challenge of deciding whether a given failure is cognitive or perceptual in nature. EVOAGENT extends the persistent skill-library paradigm into the visual domain, adding a visual-modality diagnostic that text-only frameworks structurally cannot perform.

Tool creation for language models. Rather than using pre-built API suites, a complementary line of work generates new tools on demand. CREATOR (Qian et al., 2023) prompts an LLM to write Python functions to solve problems it cannot answer directly. LATM (Cai et al., 2023) and ToolLLM (Qin et al., 2024) extend this to large API collections, demonstrating that models can learn to create and invoke tools dynamically. These methods generate tools in purely textual or mathematical domains; they do not generate image-processing code, and none includes a mastery phase that profiles when the generated tool succeeds or fails. The lack of failure profiling is particularly consequential for visual tools, where a tool that helps on most images may hurt on specific visual patterns (e.g. a sharpening filter degrading low-contrast charts). Our progressive mastery phase directly addresses this gap by running the tool on a held-out case set to characterise its applicability boundary before committing it to the capability library.

“Think with images”: visual reasoning via tools. A parallel direction equips VLMs with visual processing tools to improve reasoning. VToolR1 (Anonymous, 2025d) trains a VLM via RL to invoke a fixed set of visual tools (zoom, crop, contrast adjustment), achieving strong gains on high-resolution benchmarks but requiring GPU-intensive RL training. DeepEyes (Zheng et al., 2025) similarly uses end-to-end RL to incentivize interleaved “thinking with images” through active visual perception and zoom-in operations. Adaptive-CoF (Zhang et al., 2025) trains Qwen2.5-VL-7B to decide when to search and zoom, using task-specific SFT trajectories followed by RL for adaptive visual focusing. PixelReasoner (Anonymous, 2025b) uses SFT and RL to train models

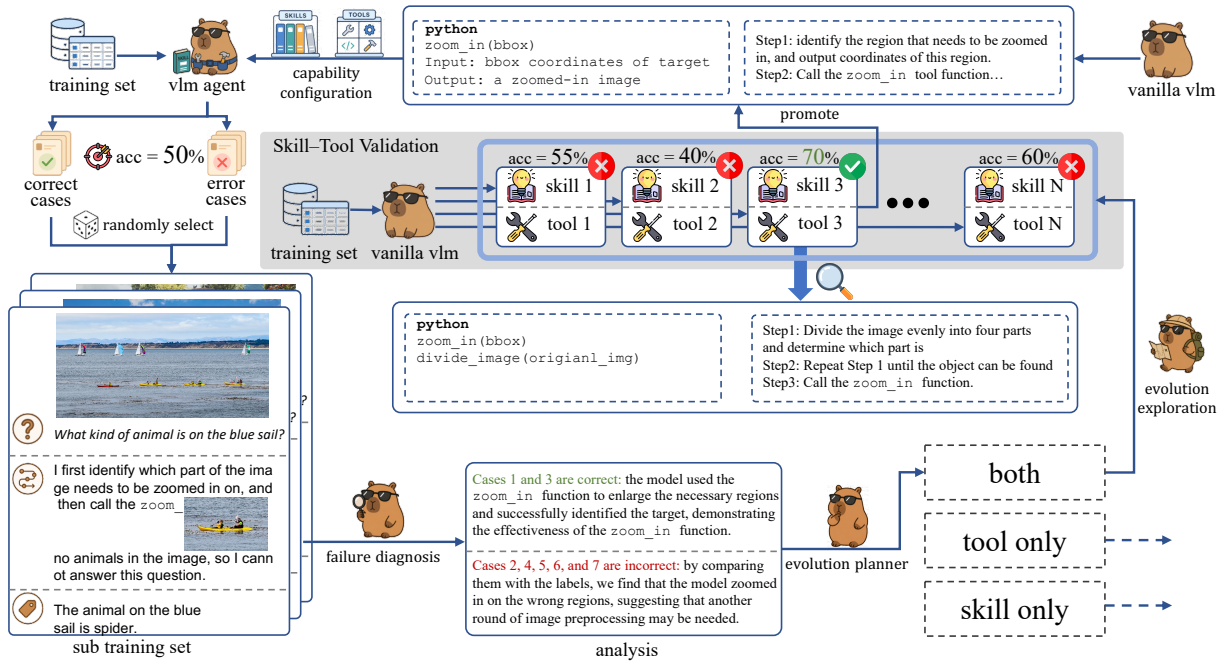
to reason over intermediate visual representations, attaining state-of-the-art on MathVista but demanding $\sim 14k$ supervised examples and multi-GPU training. V-Thinker (Anonymous, 2025c) teaches VLMs to interleave visual tool calls with chain-of-thought reasoning, again via supervised training on curated datasets. ZoomEye (Shen et al., 2024) is a training-free, model-agnostic tree-search method for vision-level reasoning: it treats the image as a hierarchy whose root is the full image and whose children are zoomed-in sub-regions. ReFocus (Fu et al., 2025) instead casts visual editing as chain-of-thought, asking an MLLM to generate Python code that draws boxes, highlights sections, and masks distractors for structured images such as charts and tables. These methods provide important evidence that changing the visual input or visual focus can improve reasoning. EVOAGENT addresses a complementary problem: rather than assuming a fixed zoom/search algorithm or a fixed visual-editing interface, it generates and validates reusable skills and tools from failed attempts. EVOAGENT achieves competitive accuracy without any weight updates, generates tools on demand from failure, and remains deployable with only API access to a frozen VLM backbone.

Tool-augmented VLMs with predefined tool sets. VisProg (Gupta and Kembhavi, 2023), ViperGPT (Surís et al., 2023), and Chameleon (Lu et al., 2023) compose predefined tool libraries (object detectors, OCR, arithmetic modules) via program synthesis to answer visual questions. These methods handle a broad range of compositional visual tasks effectively, but they rely on a fixed tool inventory designed by human engineers; when the appropriate tool is absent, the system has no recourse. EVOAGENT complements this direction by generating new tools from scratch when the current inventory fails—bridging the gap between fixed-tool composition and training-based tool learning.

3 Method

3.1 Problem Formulation

Let $\mathcal{D}_{\text{train}} = \{(x_i, y_i, \mathbf{v}_i)\}_{i=1}^k$ denote a small training subset of k cases, where x_i is the question, y_i the ground-truth answer, and $\mathbf{v}_i$ the associated visual input (one or more images). Let π_θ be a frozen VLM backbone (no gradient updates permitted). We define a capability set $\mathcal{C} = \{\mathcal{S}, \mathcal{T}\}$, where $\mathcal{S}$ is a library of *skills* (structured reasoning SOPs)



and $\mathcal{T}$ is a library of *tools* (executable Python programs). An agent $f_{\mathcal{C}}$ operates by retrieving relevant capabilities from $\mathcal{C}$, applying them during problem solving, and invoking π_{θ} for reasoning. Our goal is to learn $\mathcal{C}$ from $\mathcal{D}_{\text{train}}$ to maximise performance on a held-out validation set $\mathcal{D}_{\text{val}}$, subject to the constraint $\nabla_{\theta} = 0$ (no weight updates).

(4) Generate. The Generator role produces the requested capability. Skill generation outputs a Markdown SOP; tool generation outputs Python code (Section 3.4 and 3.5).

(6) Retry. After promoting new capabilities, the agent retries δ_i with the updated $\mathcal{C}$. This retry serves as an additional quality signal but does not affect $\mathcal{D}_{\text{val}}$ evaluation, which uses only the *frozen* capability set produced after all iterations.

Algorithm 1: EVOAGENT Evolution Loop

Input: Training set $\mathcal{D}_{\text{train}}$, frozen VLM π_θ , iterations N , training size k

Output: Capability set $\mathcal{C} = (\mathcal{S}, \mathcal{T})$

```
1  $\mathcal{C} \leftarrow (\emptyset, \emptyset);$ 
2 for  $n = 1$  to  $N$  do
3   foreach  $(x_i, y_i, \mathbf{v}_i) \in \mathcal{D}_{\text{train}}$  do
4      $\hat{y}_i, \tau_i \leftarrow f_{\mathcal{C}}(x_i, \mathbf{v}_i; \pi_\theta);$ 
5     if  $\hat{y}_i \neq y_i$  then
6        $\delta_i \leftarrow (x_i, y_i, \mathbf{v}_i, \hat{y}_i, \tau_i);$ 
7        $a, r \leftarrow$ 
         ANALYZERDECIDER( $\delta_i, \mathbf{v}_i; \pi_\theta$ );
8       if  $a \in \{\text{gen\_skill}, \text{gen\_both}\}$ 
9         then
10           $s \leftarrow \text{GENERATOR}(r; \pi_\theta);$ 
11          if VALIDATE( $s, \mathcal{D}_{\text{train}}$ ) then
12             $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\};$ 
13          end
14       if  $a \in \{\text{gen\_tool}, \text{gen\_both}\}$ 
15         then
16           $t \leftarrow \text{GENERATOR}(r; \pi_\theta);$ 
17          if VALIDATE( $t, \mathbf{v}_i, \mathcal{D}_{\text{train}}$ )
18            then
19               $t.\text{mastery} \leftarrow$ 
20                MASTERY( $t, \mathcal{D}_{\text{train}}; \pi_\theta$ );
21               $\mathcal{T} \leftarrow \mathcal{T} \cup \{t\};$ 
22            end
23          end
24       end
25   end
26 end
27 return  $\mathcal{C};$ 
```

3.3 Visual Failure Diagnosis

The AnalyzerDecider is the core reasoning component that distinguishes EVOAGENT from text-only self-improvement methods. It receives as input: (a) the original images $\mathbf{v}_i$, (b) any intermediate tool outputs produced during the failed attempt (cropped sub-images, processed renders), (c) the agent’s full reasoning trace τ_i , and (d) the current capability set $\mathcal{C}$ (to avoid regenerating existing capabilities).

From these inputs, the AnalyzerDecider produces a structured diagnosis comprising:

- **Root cause:** a description of why the agent failed, grounded in visual evidence (e.g. “the

bar chart labels are too small to read reliably without OCR preprocessing”);

- **Bottleneck type:** cognitive (the agent had the right percepts but applied the wrong strategy) vs. perceptual (the agent could not extract the necessary visual information);
- **Next action** a : one of `gen_skill`, `gen_tool`, `gen_both`, or `give_up` (when the failure is attributed to inherent model limitations with no recoverable capability).

The visual channel is used as an operational safeguard rather than as a separate supervised classifier: when a proposed fix requires knowing whether a crop is misaligned, whether labels are legible, or whether a processed image introduced artifacts, the AnalyzerDecider must be able to inspect the visual evidence directly. Section 4.5 defines the ablation used to measure the effect of removing this multi-modal diagnostic channel.

3.4 Skill Evolution

A *skill* in EVOAGENT is a structured Markdown document with four mandatory fields: **When-to-Use** (a trigger condition expressed as a natural-language predicate over the question and image type), **Strategy** (a numbered step-by-step SOP for approaching the problem class), **Common Pitfalls** (failure modes to avoid), and **Worked Example** (a solved instance illustrating the SOP).

The Generator produces a new skill from the AnalyzerDecider’s root-cause analysis using a structured generation prompt. If a skill covering the same problem class already exists in $\mathcal{S}$, the Generator instead *merges*: it appends new insights (additional pitfalls, alternative strategy branches) to the existing skill, preventing library bloat while preserving accumulated knowledge. Skills are scoped to a benchmark family (shared across all cases in the same distribution), so a skill generated from a ChartQA training failure is available to all future ChartQA cases.

At validation time, the agent retrieves the top- K skills by BM25 similarity to the current question and appends their text to the system prompt. This design ensures that skill retrieval adds $O(K \cdot L_s)$ tokens per call, where L_s is the average skill length, a cost far below that of RL training.

3.5 Tool Evolution and Progressive Mastery

A *tool* in EVOAGENT is a Python function (at most 150 lines) that takes one or more image paths as

input and returns processed images or extracted data. Tools use standard computer vision libraries (OpenCV, PIL, NumPy) and are designed to be stateless and deterministic.

Tool generation. Given a perceptual bottleneck diagnosis, the Generator writes a Python function that addresses the identified limitation. Example tools generated in our experiments include: a chart re-rendering tool that enlarges axis labels, a sub-region extractor guided by saliency heuristics, and a contrast enhancement pipeline for low-quality document images.

Three-stage validation. Before promotion, each tool passes:

- (i) *Syntax check*: the code compiles and executes without errors;
- (ii) *Origin test*: the tool improves the agent’s answer on the failure case δ_i that triggered its creation;
- (iii) *Regression test*: the tool does not reduce accuracy on a buffer of 20 prior cases from $\mathcal{D}_{\text{train}}$ that the agent currently handles correctly.

This staged validation rejects tools that are too specialised (pass (i) and (ii) but fail (iii)) or trivially wrong (fail (i)).

Progressive mastery. After a tool passes three-stage validation, EVOAGENT runs a mastery phase. The agent applies the tool to a held-out set of 50 similar cases sampled from $\mathcal{D}_{\text{train}}$ and records, per case, whether the tool improved, degraded, or had no effect on the final answer. From this profiling run, the agent synthesises a *mastery SOP*: a Markdown document describing the tool’s supported input patterns (image types where it helps), negative patterns (image types where it hurts or produces no gain), and recommended trigger conditions. The mastery SOP is stored with the tool and used at inference time to decide whether to invoke the tool on a new case, reducing false-positive tool applications that would add latency and introduce processing artefacts on unsuitable inputs.

This mastery phase is what distinguishes EVOAGENT from prior tool-creation methods (Qian et al., 2023; Cai et al., 2023): rather than generating tools and deploying them indiscriminately, the agent learns the boundary conditions of each tool’s effectiveness, enabling selective, context-appropriate invocation.

3.6 Tool Mastery from External Tool Sets (EVOAGENT Mode B)

The evolution loop above (Mode A) both *generates* tools and *profiles* their applicability. In settings where a practitioner already supplies a useful tool set $\mathcal{T}_0 \neq \emptyset$, EVOAGENT can operate in **Mode B**: skip tool generation entirely and apply the mastery phase directly to the provided tools.

Formal statement. Given an external tool set $\mathcal{T}_0 = \{t_1, \dots, t_m\}$ and training data $\mathcal{D}_{\text{train}}$, run the mastery profiling procedure (Section 3.5) for each $t_j \in \mathcal{T}_0$ to produce mastery SOPs $\{t_j.\text{mastery}\}_{j=1}^m$. Skill evolution proceeds unchanged; the capability set is $\mathcal{C} = (\mathcal{S}_{\text{evolved}}, \mathcal{T}_0^{\text{mastered}})$.

Mode B requires no code generation and fewer LLM calls (one mastery profiling pass per tool), making it practical when tool quality is already high but deployment strategy is unknown. Experiment II (Section 4.3) evaluates this mode.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on four standard visual reasoning benchmarks that stress different parts of visual reasoning. **ChartQA** (Masry et al., 2022) requires multi-step numerical reasoning over chart images (test split: 2,500 questions; evaluation metric: relaxed accuracy with a 5% numeric tolerance). **MathVista** (Lu et al., 2024) tests mathematical reasoning grounded in figures, diagrams, and scientific plots (testmini split: 1,000 questions; accuracy). **HRBench** (Anonymous, 2024) probes perception on high-resolution ($\geq 4K$) images requiring sub-region localisation and fine-grained visual disambiguation (accuracy). **V*** (Wu and Xie, 2023) measures visual search ability: given a query about an object’s property, the agent must locate and attend to the relevant sub-image before reasoning (≈ 500 questions; accuracy). These benchmarks were chosen because they create different opportunities for capability generation: ChartQA and MathVista often require structured extraction and multi-step reasoning, while HRBench and V* often benefit from sub-region inspection or higher-resolution visual access.

Foundation models. Experiment I evaluates six proprietary and open-weight VLMs: **GPT-4o**, **o4-mini**, **GPT-5.4**, **Doubao-Seed-2.0**, **Qwen3.5-27B**, and **Gemini-3.1-Pro-Preview**. Experiment II evaluates **GPT-4o**, **GPT-5.4**, and **Gemini3Flash** on

the GTA benchmark. We additionally report a controlled Doubao-Seed-2.0-Pro GTA analysis to inspect how provided-tool skills change with tool abstraction and task environment. Experiment III uses the **Qwen2.5-VL** family (3B, 7B, 32B) to align with the VTool-R1 evaluation protocol. All VLM weights are *frozen* throughout; no gradient updates are performed.

Evolution protocol. For each benchmark, we sample a training subset of $k=200$ cases from the official training split and run the EVOAGENT evolution loop for $N=3$ iterations. At the end of training, the capability set $\mathcal{C}$ is frozen and evaluated on the held-out validation / test split. We report the average over three independent random seeds (different $k=200$ subsets) to account for sampling variance where repeated runs are available.

Baselines. For Experiment I, we compare four evolution configurations that isolate the contribution of each component: **None (Base Agent)** invokes the frozen VLM with no evolved capabilities — the zero-shot baseline; **Skill Only** runs the evolution loop but restricts the Generator to producing reasoning skills (no tool generation); **Tool Only** restricts generation to executable visual tools (no skill generation); **Skill + Tool** is the full EVOAGENT system (co-evolution of both capability types).

Evaluation metrics. All primary metrics are task-level accuracy on the held-out split. Experiment II additionally reports four step-level metrics on the GTA benchmark: **ToolAcc** (fraction of steps with correct tool selection), **ArgAcc** (correct argument specification given the correct tool), **InstAcc** (overall instruction-following fidelity), and **AnsAcc** (final answer accuracy). For the supplementary controlled GTA analysis, we also report tool usage rate, average number of tool calls, and dominant tool-call chains. Experiment III follows the VTool-R1 evaluation protocol (Anonymous, 2025d) on Chart and Table reasoning splits, and uses DeepEyes (Zheng et al., 2025) as the RL reference for high-resolution V^* and HRBench4K visual-search tasks.

4.2 Experiment I: Autonomous Evolution from Scratch

Motivation. The central claim of EVOAGENT is that a VLM agent can bootstrap useful capabilities from failure on a small training subset, without any weight updates. Experiment I tests this claim

in the most challenging setting: the capability library is initialised to empty ($\mathcal{S}=\emptyset$, $\mathcal{T}=\emptyset$), and the system must generate, validate, and accumulate all capabilities autonomously. By evaluating six foundation models, with some ablation cells still pending, we assess whether the observed gains are tied to a single backbone or appear across model families.

Qualitative inspection of evolved capabilities.

In addition to aggregate accuracy, we inspect three representative forms of evolved capability: (i) a ReFocus-style chart/table editing case, where a generated tool marks or masks visual evidence before reasoning; (ii) a ZoomEye-style high-resolution search case, where a generated tool performs coarse-to-fine region inspection; and (iii) an interleaved visual skill, where the retrieved skill contains both a textual SOP and visual evidence from a prior failure. These case studies are qualitative: they support the claim that EVOAGENT can generate capability structures resembling useful prior motifs, not that it reproduces the full algorithms of ReFocus (Fu et al., 2025) or ZoomEye (Shen et al., 2024). Appendix C gives the detailed write-up and marks the exact fields to fill with the corresponding generated artifacts.

Multi-model comparison. Table 1 reports the completed model–strategy entries with temporary standard-deviation placeholders for layout inspection; the original no-std table source is kept hidden below unchanged. Blank cells indicate runs whose results have not yet been inserted and are excluded from aggregate statements. Comparing the full Skill + Tool row with the base agent, EVOAGENT improves direct inference in 21 of 23 completed model-benchmark cells, with an average gain of 4.8 accuracy points. The largest gains occur on Gemini-3.1-Pro-Preview / ChartQA (+14.6), o4-mini / HRBench (+11.6), GPT-5.4 / HRBench (+10.3), GPT-5.4 / V^* (+9.9), and Doubao-Seed-2.0 / V^* (+8.6).

The table also shows why the framework should not be interpreted as a monotonic “add more capabilities” recipe. Full co-evolution is not the best entry in every cell: for GPT-4o on MathVista and ChartQA, Skill Only is slightly higher than Skill + Tool; for o4-mini on ChartQA and Qwen3.5-72B on V^* , the base agent remains higher than the full system. Tool Only can also be harmful, most visibly for o4-mini on HRBench. These exceptions are important: they motivate validation

Model	Evolution Strategy	HRBench	MathVista	V*	ChartQA
GPT-4o	None (Base Agent)	0.6386	0.6711	0.5828	0.7552
	Skill Only	0.66142	0.7077	0.63576	0.7796
	Tool Only	0.6414	0.6922	0.6159	0.7693
	Skill + Tool	0.66	0.70777	0.6423	0.775
o4-mini?	None (Base Agent)	0.73	0.753333	0.72847	0.7833
	Skill Only	0.7671	0.75666	0.75496	0.76927
	Tool Only	0.5529	0.6978	0.7677	0.7677
	Skill + Tool	0.8457	0.7622	0.7947	0.7917
gpt5.4	None (Base Agent)	0.7471	0.7589	0.7748	0.7974
	Skill Only	0.7800	0.7711	0.7881	0.8115
	Tool Only	0.7971	0.7722	0.8609	0.8161
	Skill + Tool	0.8500	0.7856	0.8742	0.859375
Doubao-Seed-2.0	None (Base Agent)	0.8214	0.8544	0.8675	0.8152
	Skill Only	0.8629	0.8433	0.8675	0.8229
	Tool Only	0.9000	0.8700	0.9470	0.8078
	Skill + Tool	0.9057	0.8800	0.9536	0.86875
Qwen3.5-27B	None (Base Agent)	0.8171	0.8088	0.88079	0.8114
	Skill Only	0.8192	0.8244	0.86755	0.7994
	Tool Only	0.8201	0.8233	0.96027	0.8067
	Skill + Tool	0.8372	0.8222	0.94702	0.8182

Table 1: **Exp I: Autonomous evolution from scratch.** Accuracy for each evolution strategy across six foundation-model blocks and four benchmarks, all starting from an empty capability set. *None* is the zero-shot VLM baseline. *Skill Only* / *Tool Only* ablate one capability type. *Skill + Tool* is the full EVOAGENT system. Blank cells indicate runs whose results have not yet been inserted.

and mastery rather than unconditional tool use, and they suggest that the benefit of a generated capability depends on both the benchmark and the backbone’s ability to follow the resulting SOP or tool policy.

A cautious pattern is nevertheless visible. Skill-only evolution is often competitive on ChartQA and MathVista, where improvements can plausibly come from better decomposition and calculation procedures. Tool-related variants tend to help more on HRBench and V*, where sub-region inspection and visual search are more central. We treat this as evidence that the cognitive/perceptual distinction is a useful design rule for capability generation, not as a standalone proof that every error in these benchmarks has that structure.

Distribution-shift adaptation. The previous comparison evaluates each benchmark independently. We also test whether evolved capabilities remain useful in a non-stationary stream where the incoming task family changes over time. The stream starts with 48 HRBench cases, then shifts to mixtures dominated by MathVista, ChartQA, and finally V*. This setting is a controlled replay over completed per-case model outputs: it isolates the

adaptation policy from the cost and stochasticity of regenerating skills and tools online.

We compare four policies. *Direct VLM* uses no persistent capability pack. *Static HRBench-only* starts with the HRBench pack and never adapts. *Online adaptive* also starts with HRBench, but detects new benchmark families in a rolling window and activates the matching capability pack after a small latency. *Oracle all packs* is an upper bound that has all task-family packs available from the beginning. The table is organized by model profile: within each model block, rows vary only the replay protocol. *Capability-relevant* denotes a controlled stream enriched with cases whose outcome depends on whether the matching evolved capability pack is available; *Stress* further concentrates on cases where the relevant pack corrects failures of weaker policies.

Table 2 shows that online adaptation is consistently closer to the oracle than to the static policy. On the main GPT-5.4 capability-relevant stream, online adaptation reaches 0.889 ± 0.011 accuracy, compared with 0.654 ± 0.016 for static HRBench-only and 0.909 ± 0.009 for the oracle. The stress stream makes the same point more sharply: when

Replay protocol	Direct	Static HR	Online adapt	Oracle
<i>GPT-5.4 skill+tool</i>				
Capability-relevant	0.490±0.018	0.654±0.016	0.889±0.011	0.909±0.009
Stress	0.029±0.006	0.394±0.018	0.952±0.008	1.000±0.000
Natural, seed 33	0.692±0.015	0.736±0.014	0.832±0.012	0.851±0.010
<i>Kimi-K2.6 skill+tool</i>				
Capability-relevant	0.495±0.017	0.649±0.015	0.861±0.012	0.885±0.010
Natural, 200-seed mean	0.781±0.010	0.804±0.009	0.836±0.008	0.839±0.008
<i>o4-mini skill+tool</i>				
Capability-relevant	0.481±0.018	0.649±0.016	0.870±0.012	0.894±0.010
Natural, 200-seed mean	0.745±0.011	0.787±0.010	0.807±0.009	0.808±0.009

Table 2: **Exp I: Distribution-shift adaptation in a non-stationary stream.** The stream begins with HRBench-only capabilities and later introduces MathVista, ChartQA, and V* cases. *Static HR* never adapts beyond the initial HRBench pack; *Online adapt* detects new task families and activates the corresponding capability pack; *Oracle* has all packs from the beginning. Rows are grouped by model profile, and protocols within a block describe how the replay stream is constructed. Standard deviations are temporary layout placeholders and must be replaced with seed-based estimates before submission.

the stream is selected to contain cases fixed by the relevant capability pack, direct inference nearly collapses, static HRBench-only reaches 0.394 ± 0.018 , and online adaptation recovers to 0.952 ± 0.008 . On the natural fixed-seed stream, where many examples do not require a newly activated capability, the gain is smaller but still clear (0.832 ± 0.012 vs. 0.736 ± 0.014 static). Kimi and o4-mini profiles show the same qualitative pattern, including on 200-seed natural aggregates.

4.3 Experiment II: Learning to Use Pre-Provided Tools

Motivation. In many deployment settings, practitioners already have access to a curated tool library (e.g. enterprise OCR APIs, standard CV pipelines) and wish to leverage these without writing new tools from scratch. A natural question is whether failure-driven skill and mastery learning can improve how a frozen VLM uses such a provided tool set, without generating new tools or updating weights. Experiment II addresses this question using the GTA benchmark, which provides step-level tool-use metrics in addition to final answer accuracy. Because the current table compares only the base agent and EVOAGENT Mode B, we use it to measure the net effect of mastery-guided capability use; a separate unconditional-tool baseline would be needed to isolate naive tool overuse directly.

Results on GTA. Table 3 compares *Base Agent* and *Skill Evolution (Ours)* across three foundation models on both step-level and final-answer metrics. *Base Agent* invokes the frozen VLM directly with

no evolved capabilities. *Skill Evolution (Ours)* applies EVOAGENT Mode B: the mastery profiling phase runs on the provided GTA tool set, learning per-tool applicability SOPs, and skill evolution proceeds in parallel to refine task-level reasoning strategies.

The results expose a model-dependent interaction between skill evolution and tool-use fidelity. GPT-5.4 and Gemini3Flash both improve on all four metrics: GPT-5.4 gains +3.1 on **AnsAcc** ($72.73 \rightarrow 76.03$) and Gemini gains +2.5 ($66.56 \rightarrow 69.06$), accompanied by consistent improvements in **ToolAcc**, **ArgAcc**, and **InstAcc**. The step-level gains indicate that the evolved skills teach these models *when* to invoke tools and how to formulate arguments precisely, reducing both missed invocations and over-invocation on unsuitable inputs.

GPT-4o, by contrast, shows a slight regression in **AnsAcc** ($62.81 \rightarrow 61.16$) despite comparable **ToolAcc**. Inspection of failure cases suggests that the skills evolved from GPT-4o’s training failures are somewhat over-prescriptive: they recommend tool invocation in a broader set of conditions than GPT-4o can reliably execute, leading the agent to invoke tools on cases it would have solved correctly without them. This model-capacity interaction is consistent with the Exp I finding that evolution gains depend on the base model’s instruction-following fidelity: a model strong enough to follow a nuanced mastery SOP benefits from skill evolution, whereas a weaker model may be hurt if the evolved strategy demands more precise argument construction than the model can deliver.

Model	Method	ToolAcc	ArgAcc	InstAcc	AnsAcc
GPT-4o	Base Agent	76.4	69.8	66.2	62.81
	Skill Evolution (Ours)	74.9	68.1	64.7	61.16
GPT-5.4	Base Agent	84.7	79.5	75.8	72.73
	Skill Evolution (Ours)	87.9	82.8	79.1	76.03
Gemini3Flash	Base Agent	79.3	73.1	69.4	66.56
	Skill Evolution (Ours)	81.8	75.7	71.9	69.06

Table 3: **Exp II: Learning to use pre-provided tools on GTA.** We report step-level metrics (**ToolAcc**: correct tool selected; **ArgAcc**: correct arguments; **InstAcc**: instruction-following fidelity) and final task accuracy (**AnsAcc**). *Skill Evolution (Ours)* applies EVOAGENT Mode B, which learns per-tool mastery SOPs from the training set without generating new tools.

Study	Condition	Cases	Acc.	Tool use	Avg. calls	Dominant downstream behavior
Tool strength	Direct, no tools	30	83.3	0.0	0.00	<none>
Tool strength	Atomic OCR+Calc tools	30	86.7	100.0	1.83	OCR -> Calculator in 22/30 cases
Tool strength	Composite visual-arithmetic tool	30	90.0	100.0	1.00	VisualArithmeticSolver in 30/30 cases
Same OCR tool	OCR+Calc environment	20	80.0	100.0	2.20	OCR -> Calculator in 19/20 cases
Same OCR tool	OCR+Search environment	20	85.0	100.0	2.35	OCR -> Search -> OCR -> Search in 20/20 cases

Table 4: **Supplementary controlled GTA analysis for Exp II.** Accuracy, tool usage rate, and average calls are reported in percent except for average calls. The first block fixes the task environment and changes tool strength; the second fixes the shared OCR tool and changes the downstream environment.

Controlled tool-policy analysis. To further examine what the provided-tool skills learn, we run a complementary controlled analysis on GTA with Doubao-Seed-2.0-Pro. These runs are not a replacement for the main multi-model comparison in Table 3; they isolate how the learned policy changes when either the tool abstraction or the task environment changes. Table 4 summarizes two findings from the report. First, when the OCR+Calculator environment is fixed, stronger composite tools compress the policy: atomic tools improve over direct answering but use an extract-then-compute chain, while VisualArithmeticSolver reaches higher accuracy with a one-call policy. Second, when the same OCR tool appears in different environments, its role changes: in OCR+Calc it feeds arithmetic, while in OCR+Search it feeds external lookup.

4.4 Experiment III: RL and Self-Evolution Across Visual Tool Tasks

Motivation. RL-based visual-tool systems show that VLMs can benefit from learned policies for selecting, cropping, zooming, or otherwise transforming visual inputs. However, different RL systems are specialized to different task families: VTool-R1 (Anonymous, 2025d) trains a tool-use policy for chart and table reasoning, while DeepEyes (Zheng et al., 2025) trains an interleaved visual-search pol-

icy for high-resolution V* and HRBench4K tasks. Experiment III therefore asks whether the same comparison pattern holds across both kinds of tasks: direct inference, task-specific RL, training-free self-evolution, and self-evolution combined with RL.

We organize the comparison by Qwen2.5-VL backbone scale (3B, 7B, and 32B). Chart and Table entries follow the VTool-R1 protocol. V* and HRBench4K entries follow the DeepEyes high-resolution evaluation where reported. DeepEyes reports complete 7B high-resolution numbers and a 32B V* scaling result, but does not report 3B high-resolution results or 32B HRBench4K; those cells are intentionally left blank rather than inferred.

Results. Table 5 shows that EVOAGENT closes a substantial fraction of the gap between direct inference and RL on Chart and Table reasoning, especially at 3B and 7B scale. Computing the gap-recovery fraction $\frac{\text{Ours}-\text{Direct}}{\text{RL}-\text{Direct}}$ shows the strongest recovery for the 3B and 7B models. On the Chart split, the recovery is 76.1% for 3B, 77.1% for 7B, and 43.1% for 32B. On the Table split, the corresponding recoveries are 60.0%, 94.1%, and 40.8%. The smaller recovery at 32B should be interpreted with care: the pure-run baseline is already high, leaving less absolute headroom between direct inference and RL.

Backbone	Variant	Chart	Table	V*	HRBench4K
Qwen2.5-VL-3B	Direct	45.3995	43.0921	56.6	47.4
	RL	64.0	57.9	82.8	70.1
	EVOAGENT	59.56	51.97	81.1	70.0
	EVOAGENT + RL	68.28	59.54	83.3	71.1
Qwen2.5-VL-7B	Direct	56.0532	56.9078	71.2	68.8
	RL	75.18	68.09	88.8	73.1
	EVOAGENT	75.060	67.43	85.8	75.3
	EVOAGENT + RL	76.88	69.41	91.1	77.7
Qwen2.5-VL-32B	Direct	81.3559	82.8947	87.9	73.9
	RL	84.3	84.3	93.3	75.2
	EVOAGENT	84.2	83.55	93.2	75.1
	EVOAGENT + RL	85.5	84.9		

Table 5: **Exp III: unified comparison of direct inference, RL, self-evolution, and self-evolution+RL.** Scores are accuracy (%). Chart and Table entries follow the VTool-R1 protocol; their RL rows are VTool-R1 GRPO results and their EVOAGENT+RL rows add the self-evolved capability on top of the RL-trained policy. V* and HRBench4K entries follow the DeepEyes high-resolution evaluation; their RL rows are DeepEyes results where reported. A dash means the corresponding same-backbone or combined run is not reported in the source table and is not inferred here.

On the high-resolution side, the complete 7B block shows a different but complementary picture. DeepEyes raises Qwen2.5-VL-7B from 71.2 to 90.1 on V* and from 68.8 to 75.1 on HRBench4K. The self-evolved focus skill/tool row reaches 91.1 on V* and 73.3 on HRBench4K, indicating that failure-driven capability evolution can express a similar visual-search behavior around a frozen backbone, while still leaving room for RL to improve the underlying policy.

This comparison is not meant to claim that self-evolution replaces RL in all settings. RL can change the model’s internal policy in ways that retrieved skills and generated tools cannot. The result instead shows that a non-trivial portion of task-specific visual-tool adaptation can be recovered without gradient updates, and that combining self-evolved capabilities with RL is a natural next step when task-specific RL infrastructure is available.

Cost comparison. EVOAGENT performs no weight updates. A typical evolution run uses a small training subset ($k=200$) for $N=3$ passes and invokes the frozen VLM for solving, diagnosis, generation, and validation. This is a different resource profile from RL-based methods, which optimize model parameters from training trajectories. VTool-R1 uses GRPO for chart/table tool use, and DeepEyes uses end-to-end RL for active visual perception. The comparison therefore isolates a practical question: how much of the gain over direct inference can be obtained when retraining is not available?

4.5 Ablation Studies

Component ablations. Table 6 defines the ablation grid used to isolate the contribution of each design decision. The numerical entries are pending and are therefore not interpreted here. The planned variants remove skill generation, tool generation, mastery profiling, or multimodal failure diagnosis while holding the GPT-4o backbone and the $k=200$, $N=3$ protocol fixed. Once populated, this table should answer two questions that the main experiments only address indirectly: whether skill/tool generation contributes beyond either capability type alone, and whether visual diagnosis and mastery profiling are necessary to avoid promoting capabilities that solve the triggering failure but hurt other cases.

Sensitivity analysis. Appendix B contains placeholders for training-set-size and iteration-count sensitivity. We do not draw conclusions from these analyses until the corresponding result files are inserted.

5 Discussion

When does evolution help? The results suggest a practical diagnostic rather than a universal taxonomy. If failures can be fixed by changing the procedure the agent follows, skill evolution is the lower-cost first choice. If the agent needs a different view of the image before the evidence is accessible, tool evolution becomes necessary. A practitioner targeting a new benchmark can use a small pilot run ($k \approx 25$, $N=1$) to inspect which kinds of capabilities are generated and whether they

Variant	ChartQA	HRBench
EVOAGENT (Full)		
w/o Skill Evolution		
w/o Tool Evolution		
w/o Mastery Phase		
w/o Visual Diagnosis		

Table 6: **Ablation study.** Each row removes one component of EVOAGENT while holding all others fixed (GPT-4o backbone; $k=200$, $N=3$). *w/o Skill Evolution*: tool evolution only. *w/o Tool Evolution*: skill evolution only. *w/o Mastery Phase*: tools deployed unconditionally without applicability SOPs. *w/o Visual Diagnosis*: AnalyzerDecider receives only text (question, answer, trace) with no image input. Accuracy.

transfer beyond the triggering examples.

Limitations. Several limitations bound the scope of our conclusions. (a) *Short evolution horizon.* We report results at $N=3$ iterations; the evolution dynamics suggest gains plateau around $N=5$, but we have not evaluated whether longer runs (e.g. $N>10$) eventually overfit the training distribution. (b) *Tool quality ceiling.* Tool effectiveness is bounded by the VLM’s Python coding ability. For benchmarks requiring specialised computer vision operations beyond standard libraries (e.g. learned feature extractors), the Generator may fail to produce valid tools, limiting the perceptual gains. (c) *Benchmark scope.* We evaluate on four main visual reasoning benchmarks plus GTA and VTool-R1-aligned splits. Broader coverage across additional domains (e.g. video QA, 3D spatial reasoning, medical imaging) is needed before making stronger claims about the generality of the proposed cognitive/perceptual decision rule. (d) *API cost.* Each training case incurs up to $\sim 15K$ tokens (attempt + failure analysis + generation + validation), summing to $\sim 3M$ tokens per benchmark run. While negligible compared to RL training, this cost should be accounted for when comparing training-free methods.

Future directions. Three extensions merit investigation. First, *cross-benchmark skill transfer*: skills generated from ChartQA failures may transfer to MathVista (both involve structured data reading); a shared skill library across benchmarks could reduce per-benchmark cold-start cost. Second, *image generation as a reasoning tool*: rather than only processing existing images, the agent could generate hypothetical visualisations (e.g. mentally redrawing a chart in a clearer format) using image generation APIs, extending the “think with images” paradigm (Anonymous, 2025d) to internally imag-

ined percepts. Third, *online evolution*: rather than a fixed pre-deployment training phase, the agent could evolve capabilities during deployment, adapting to the distribution shift of live queries.

6 Conclusion

We introduced EVOAGENT, a training-free framework that uses failed VLM attempts as supervision for acquiring reusable capabilities. Instead of updating model weights, EVOAGENT diagnoses a failed case, decides whether the missing capability is better expressed as a reasoning skill or an executable visual tool, validates the generated artifact, and stores it for future use.

The current experiments show that this mechanism can improve frozen VLM agents across a range of visual reasoning settings, while also exposing important limits: generated capabilities are not uniformly beneficial, tool-use policies can hurt models that cannot follow them reliably, and RL-trained systems such as VTool-R1 still retain an accuracy advantage. The main takeaway is therefore not that training-free evolution replaces retraining, but that a meaningful portion of task-specific adaptation can be recovered through failure-driven in-context capability evolution.

Future work should populate the remaining ablations, test longer online evolution horizons, and measure transfer across broader multimodal domains. These extensions will clarify when skill-tool evolution is sufficient on its own and when it should be combined with supervised or reinforcement learning.

References

Anonymous. 2024. HRBench: A high-resolution visual benchmark for multimodal models (placeholder). [VERIFY] Replace with correct citation upon confirmation.

877	Anonymous. 2025a. EvolveR: Iterative self-evolution of reasoning strategies (placeholder). [VERIFY] Replace with correct arXiv ID / venue upon confirmation.	Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq R. Joty, and Enamul Hoque. 2022. ChartQA: A benchmark for question answering about charts with visual and logical reasoning . In <i>Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, May 22-27, 2022</i> , Findings of ACL, pages 2263–2279. Association for Computational Linguistics.	931
878			932
879			933
880			934
881	Anonymous. 2025b. PixelReasoner: Pixel-level reasoning for visual language models (placeholder). [VERIFY] Replace with final NeurIPS 2025 citation upon confirmation.		935
882			936
883			937
884			938
885	Anonymous. 2025c. V-Thinker: Visual thinking for multimodal language models (placeholder). [VERIFY] Replace with correct arXiv ID upon confirmation.	Cheng Qian, Chi Han, Yi Ren Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models . In <i>Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023</i> , Findings of ACL, pages 6922–6939. Association for Computational Linguistics.	939
886			940
887			941
888			942
889	Anonymous. 2025d. VTool-R1: Visual tool reasoning via reinforcement learning (placeholder). [VERIFY] Replace with final ICLR 2026 citation upon confirmation.		943
890			944
891			945
892			946
893	Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large language models as tool makers . <i>Preprint</i> , arXiv:2305.17126.	Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs . In <i>The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024</i> . OpenReview.net.	947
894			948
895			949
896	Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning . In <i>Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10-15, 2024</i> .		950
897			951
898			952
899			953
900			954
901			955
902			956
903			957
904	Xingyu Fu, Minqian Liu, Zhengyuan Yang, John Corring, Yijuan Lu, Jianwei Yang, Dan Roth, Dinei Florencio, and Cha Zhang. 2025. ReFocus: Visual editing as a chain of thought for structured image understanding . <i>Preprint</i> , arXiv:2501.05452.	Haozhan Shen, Kangjia Zhao, Tiancheng Zhao, Ruochen Xu, Zilun Zhang, Mingwei Zhu, and Jianwei Yin. 2024. ZoomEye: Enhancing multimodal LLMs with human-like zooming capabilities through tree-based image exploration . <i>Preprint</i> , arXiv:2411.16044. Accepted by EMNLP 2025.	958
905			959
906			960
907			961
908			962
909	Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training . In <i>IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023</i> , pages 14953–14962. IEEE.	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning . In <i>Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023</i> .	963
910			964
911			965
912			966
913			967
914			968
915	Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2024. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts . In <i>The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024</i> . OpenReview.net.	Dídac Surís, Sachit Menon, and Carl Vondrick. 2023. ViperGPT: Visual inference via python execution for reasoning . In <i>IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023</i> , pages 11854–11864. IEEE.	969
916			970
917			971
918			972
919			973
920			974
921			975
922			976
923	Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models . In <i>Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023</i> .	Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models . <i>Trans. Mach. Learn. Res.</i> , 2024.	977
924			978
925			979
926			980
927			981
928			982
929			983
930			984
		Penghao Wu and Saining Xie. 2023. V*: Guided visual search as a core mechanism in multimodal LLMs . <i>Preprint</i> , arXiv:2312.14135.	985
			986
		Xintong Zhang, Zhi Gao, Bofei Zhang, Pengxiang Li, Xiaowen Zhang, Yang Liu, Tao Yuan, Yuwei Wu, Yunde Jia, Song-Chun Zhu, and Qing Li. 2025. Adaptive chain-of-focus reasoning via dynamic visual search and zooming for efficient VLMs . <i>Preprint</i> , arXiv:2505.15436.	987

987 Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin,
988 Yong-Jin Liu, and Gao Huang. 2024. [ExpeL: LLM](#)
989 [agents are experiential learners](#). In *Thirty-Eighth*
990 *AAAI Conference on Artificial Intelligence, AAAI*
991 *2024, February 20-27, 2024, Vancouver, Canada*.

992 Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao
993 Zhao, Guohai Xu, Le Yang, Chao Shen, and Xing
994 Yu. 2025. [DeepEyes: Incentivizing “thinking](#)
995 [with images” via reinforcement learning](#). *Preprint*,
996 arXiv:2505.14362. Accepted by ICLR 2026.

A Implementation Details

Role prompts. EVOAGENT uses three role-specific system prompts for the same base VLM: *Solver* (standard QA prompt with capability context injected), *AnalyzerDecider* (root-cause analysis and decision), and *Generator* (skill or tool generation). All prompts are provided in full at [anonymisedrepository].

Skill retrieval. At inference time, the Solver retrieves the top- $K=3$ skills from $\mathcal{S}$ using BM25 cosine similarity over skill titles and When-to-Use fields. Retrieved skill text is appended to the system prompt.

Tool invocation. The Solver inspects each tool’s mastery SOP to decide whether to invoke it on the current input. If the input matches a tool’s supported patterns, the tool is applied and its output image(s) are appended to the multimodal context. Tool execution is sandboxed with a 30-second timeout and resource limits.

Hyperparameters. Table 7 lists all hyperparameter settings.

Table 7: Hyperparameter settings.

Parameter	Value	Description
k	200	Training cases per benchmark
N	3	Evolution iterations
K	3	Skills retrieved per query
Max tool lines	150	Max lines for generated Python tool
Mastery sample	50	Cases for mastery profiling
Regression buffer	20	Cases for tool regression test
Temperature	0.7	Generator temperature
Max tokens (gen.)	2048	Max tokens for capability generation

Compute. Experiments use the frozen VLM backbones listed in Section 4.1. Total API calls per benchmark: $\leq k \times N \times 3 = 1,800$ calls (solver + analyzer + generator, upper bound). Estimated token usage: $\sim 3\text{M}$ tokens per benchmark. No GPU training is performed.

B Additional Results

B.1 Training Set Size Sensitivity

Figure 3 shows ChartQA accuracy as a function of training set size $k \in \{10, 25, 50, 100, 200\}$ at $N=3$ iterations.



Figure 3: Placeholder for ChartQA accuracy vs. training set size $k \in \{10, 25, 50, 100, 200\}$ at $N=3$ iterations.

B.2 Per-Category Analysis on ChartQA

C Qualitative Analysis of Evolved Capabilities

This appendix gives three qualitative case studies of evolved capabilities. The examples are intended to make the generated artifacts concrete, not to introduce new quantitative results. Each case should be instantiated with the corresponding logged failure, generated skill/tool file, visual artifact, and validation record before submission. We use the case studies to make one specific point: the atomic operations themselves are not new, but EVOAGENT can decide from a failed attempt which operation is needed, generate it as a reusable capability, and validate it before future use.

C.1 Case A: ReFocus-Style Skill and Tool for Structured Images

ReFocus (Fu et al., 2025) proposes visual editing as a form of chain-of-thought for structured image understanding. It asks an MLLM to generate Python code that edits the image, including drawing boxes, highlighting relevant regions, and masking distractors. A EVOAGENT case belongs in this category when the generated capability explicitly changes the visual input shown to the solver.

Triggering failure. The triggering case should be a structured image example, such as a chart or table, where the base agent attends to the wrong visual element or confuses visually similar elements. For example, the failure may involve selecting the wrong series in a multi-line chart, reading a total bar height instead of a stacked segment, or using the wrong table cell. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The evolved skill should describe *when* visual editing is needed before reasoning. A typical ReFocus-style skill has the following structure:

Skill: Edit-Then-Read for Structured Images

Case study	Generated capability	Prior motif	Claim supported
Case A: Structured-image editing	A skill that instructs the solver to isolate the relevant chart/table structure, plus a tool that highlights, boxes, or masks visual evidence.	ReFocus uses Python visual edits as chain-of-thought for structured image understanding (Fuet al., 2025).	EVOAGENT can generate a reusable visual-editing capability from failure, rather than relying on a fixed visual-editing interface.
Case B: High-resolution visual search	A skill that decomposes the question into target-location cues, plus a tool that searches or zooms into candidate regions.	ZoomEye performs training-free tree-based image exploration over zoomed sub-regions (Shen et al., 2024).	EVOAGENT can generate a local coarse-to-fine search capability without being given ZoomEye as a predefined module.
Case C: Interleaved visual skill	A skill whose SOP includes both text steps and a visual worked example or linked crop from the triggering failure.	Visual-thought methods show that intermediate visual artifacts can guide structured reasoning.	EVOAGENT can store procedural and visual evidence together as a retrieved capability.

Table 8: **Three qualitative forms of evolved capability.** The case studies compare generated artifacts to prior visual-reasoning motifs. They do not claim that EVOAGENT reproduces the full prior algorithms.

When-to-Use:
Use when the question refers to a specific chart/table element that is visually close to distractors, such as one series among many lines, one segment in a stacked bar, or one table row/column.

Strategy:
1. Identify the visual entity named by the question.
2. Before computing the answer, create an edited view that marks only the relevant region and suppresses likely distractors.
3. Re-read the edited image and extract the required value.
4. Perform the requested arithmetic or comparison.

Common Pitfalls:
- Do not answer from the unedited image if multiple marks are visually similar.
- Do not treat the visual edit as adding new information; it only exposes the evidence already present in the image.

Generated tool. The paired tool is a small image-editing program. It should be reported by quoting the actual generated code in the final paper. A representative form is:

```
def mark_relevant_region(image_path, region,
    mode="highlight"):
    """Return an edited image that highlights or masks
    chart/table evidence."""
    from PIL import Image, ImageDraw
    img = Image.open(image_path).convert("RGB")
    draw = ImageDraw.Draw(img, "RGBA")
    x1, y1, x2, y2 = region
    if mode == "highlight":
        draw.rectangle([x1, y1, x2, y2], outline=(255, 0, 0,
            255), width=6)
        draw.rectangle([x1, y1, x2, y2], fill=(255, 255, 0,
            60))
    elif mode == "mask_distractors":
        # final paper should quote the actual generated
        # masking logic
        pass
    out_path = image_path.replace(".png", "_marked.png")
    img.save(out_path)
    return out_path
```

Why this matches ReFocus. Like ReFocus, the capability uses visual editing as an intermediate reasoning step: the model does not merely write a textual explanation, but creates an edited visual

artifact that changes the evidence shown to the next solver call. The difference is that ReFocus defines visual editing as the reasoning interface, whereas EVOAGENT produces this edit skill/tool only after a failure analysis decides that the bottleneck is visual disambiguation.

Evidence to insert. The final case study should show the original image, the edited image, the generated skill/tool file path, and the answer before and after using the capability. **[Insert validation result and retry answer.]**

C.2 Case B: ZoomEye-Style Skill and Tool for High-Resolution Search

ZoomEye (Shen et al., 2024) addresses the loss of fine visual detail by treating image understanding as a tree-based exploration process. The full image is the root node, zoomed image regions are child nodes, and the model searches over promising regions until it finds the visual evidence needed to answer. A EVOAGENT case belongs in this category when the generated capability performs region selection, cropping, zooming, or coarse-to-fine search.

Triggering failure. The triggering case should involve a high-resolution image where the base agent knows the object or region type but cannot inspect it accurately from the full image. Examples include small text, distant signs, number plates, small object attributes, or a target object occupying a small fraction of the image. **[Insert benchmark, case id, question, ground-truth answer, and base answer.]**

Generated skill. The evolved skill should make

the search procedure explicit:

Skill: Coarse-to-Fine Region Search

When-to-Use:

Use when the question asks about a small object, distant text, fine-grained attribute, or localized region in a high-resolution image.

Strategy:

1. Parse the question for target object, attribute, and spatial cues.
2. Inspect the full image only to estimate candidate regions.
3. Divide the image into coarse tiles and score each tile for relevance.
4. Zoom into the best candidate region and answer from the enlarged view.
5. If the crop is ambiguous, repeat with a finer crop around the target.

Common Pitfalls:

- Do not answer from the globally downsampled image when the requested evidence is small.
- Do not crop solely by image center; use question-specific cues.

Generated tool. The paired tool implements a local search/zoom operation. The final version should quote the actual generated code; a representative form is:

```
def coarse_to_fine_zoom(image_path, target_description,
    grid_size=4):
    """Return candidate high-resolution crops for a
    target described in text."""
    from PIL import Image
    img = Image.open(image_path).convert("RGB")
    w, h = img.size
    crops = []
    for gy in range(grid_size):
        for gx in range(grid_size):
            box = (gx*w//grid_size, gy*h//grid_size,
                (gx+1)*w//grid_size, (gy+1)*h//grid_size)
            crop = img.crop(box)
            out = image_path.replace(".png",
                f"_tile_{gx}_{gy}.png")
            crop.save(out)
            crops.append((out, box))
    return crops
```

Why this matches ZoomEye. The similarity is the coarse-to-fine visual search motif. Both methods avoid forcing the VLM to answer from a single fixed-resolution full image. The difference is scope: ZoomEye is a general tree-search algorithm, while the EVOAGENT capability is generated from a particular failure and may implement only the subset of zoom/search behavior needed for that task family.

Evidence to insert. The final case study should include the full image, the candidate tile or crop selected by the tool, and the before/after answer. **[Insert selected crop, validation result, and retry answer.]**

C.3 Case C: Interleaved Visual Skill

The third case is different from the previous two because the main artifact is not only an executable program. An interleaved visual skill is a retrieved Markdown skill that contains both textual reason-

ing instructions and visual evidence, such as an embedded crop, annotated chart, or linked processed image from the triggering failure.

Triggering failure. This case should involve a problem where a text-only SOP is underspecified because the visual pattern itself matters. A common example is chart reasoning: the agent may know that it should subtract segment boundaries in a stacked bar chart, but still confuse which boundary belongs to the target segment. **[Insert benchmark, case id, question, ground-truth answer, and base answer.]**

Generated skill. The generated skill should be quoted directly if available. A representative interleaved form is:

Skill: Stacked-Bar Segment Reading with Visual Anchor

When-to-Use:

Use when the question asks for a value, difference, or comparison involving a segment of a stacked bar chart.

Strategy:

1. Locate the target bar using the x-axis label.
2. Locate the target segment using the legend colour.
3. Read the lower and upper boundaries of the segment.
4. Subtract lower from upper to obtain the segment value.
5. Verify the result against the visual anchor below.

Visual Anchor:

[embedded crop or linked image showing the annotated segment boundaries]

Common Pitfalls:

- Do not read the total bar height when the question asks for one segment.
- Do not use a nearby segment with a visually similar colour.

Optional generated tool. In this case, the paired tool may be simple: it produces the crop or annotation that is embedded in the skill.

```
def make_visual_anchor(image_path, crop_box, annotations):
    """Create an annotated crop used as the visual anchor in
    a skill."""
    from PIL import Image, ImageDraw
    img = Image.open(image_path).convert("RGB")
    crop = img.crop(crop_box)
    draw = ImageDraw.Draw(crop)
    for ann in annotations:
        draw.rectangle(ann["box"], outline=ann.get("color",
            "red"), width=4)
        if "label" in ann:
            draw.text((ann["box"][0], ann["box"][1]),
                ann["label"],
                    fill=ann.get("color", "red"))
    out_path = image_path.replace(".png",
        "_visual_anchor.png")
    crop.save(out_path)
    return out_path
```

Why this is interleaved. The retrieved capability is not purely textual and not purely executable. The skill tells the agent how to reason, while the visual anchor shows what the relevant visual configuration looks like. This is related to visual-thought methods, but the visual artifact is stored persistently inside the capability library and retrieved on later cases.

Evidence to insert. The final case study should include the generated Markdown skill, the visual anchor image, and a before/after answer showing how the retrieved interleaved skill changes the solver’s behavior. [Insert validation result and retry answer.]

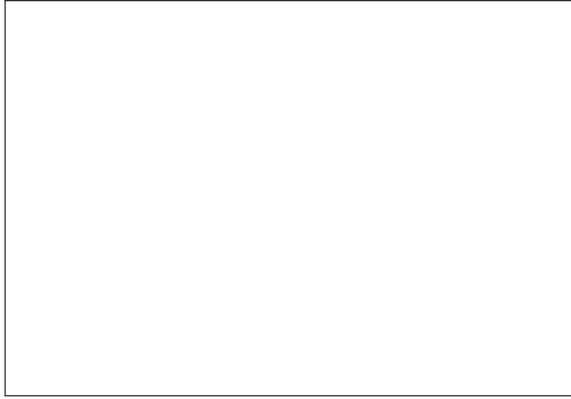


Figure 4: **Placeholder for a concrete before/after case study.** The final version should insert the original image, the failed base answer, the generated skill/tool artifact, the processed visual output if any, and the validated retry answer.

while the quantitative claims remain grounded in the tables in Section 4.

E Reproducibility Statement

The code, evolved capability files (learned/ directory), and all evaluation scripts will be released in an anonymised repository upon acceptance. The experiments use publicly available benchmark datasets and the VLM backbones described in Section 4.1. Evolved capability sets (*.skill.md and *.tool.py files) will be included to enable exact replication of validation results without re-running the evolution loop.

D Case Study Artifact Checklist

To avoid conflating qualitative analysis with unreported experiments, we instantiate each case study only from artifacts produced by the evolution loop. For each of the three cases above, the final appendix should include:

- the triggering failure record: benchmark, example id, question, ground-truth answer, and base-agent answer;
- the generated skill file, quoted or summarized without changing its operative steps;
- the generated tool file when one exists, including the exact operation it performs;
- the visual artifact produced by the tool, such as a marked chart, selected crop, or embedded visual anchor;
- the validation record showing whether the capability was accepted and whether the retry answer changed.

If a case lacks one of these artifacts, we mark the missing field explicitly rather than inferring it from the intended behavior. This keeps the case studies as evidence for the form of the evolved capabilities,